

DataForge .pathway x rime

DataForge 2026: Pathway Track

Explain the Frontier

AI research moves faster than the material used to teach it. Ideas often appear across many papers before there is a clear, reusable way to learn them. The NeurIPS 2026 Education Track was built around this gap: it asks contributors to distill emerging AI ideas into accessible resources such as interactive demos, notebooks, code, short videos, slides, lecture notes, animations, and visual explanations, along with the intended audience, prerequisites, learning objectives, and recent papers that show the concept is active now.

This Pathway track brings that challenge to DataForge. Choose one approved concept. Build an explainer that makes one difficult technical idea genuinely understandable. Connect it to Dragon Hatchling (BDH) or BDH CQ. The learner should not just read or watch. The learner should interact with the system, change something meaningful, and observe the concept behaving.

Every submission must also contain a substantial, technically correct section on BDH or BDH CQ.

The goal is simple: make one frontier AI concept click.

About Pathway

Pathway is a frontier AI company working on architectures and models for memory, latent reasoning, continual learning, and long-horizon reasoning.

Its central research program is [Dragon Hatchling, or BDH](#): a brain-inspired Post Transformer architecture family. [BDH CQ](#) is a later system in the same family that lets the model learn from demonstrations and reason without a written chain of thought.

Approved topics

These topics were pulled from concept patterns across recent NeurIPS papers and related AI venues: ideas that are emerging, resurging, or repeatedly used by researchers right now. A valid concept can be a new technical advance, or an older idea returned in an important new form, but a generic introduction to a well-established topic isn't enough. Explain why it matters now, and show its use in recent research.

Choose one topic from the list below. A team may combine two closely related topics only when the final artifact still has one central claim and one coherent learning journey.

Memory and state. Neural state space models. Linear attention. Synaptic plasticity as short-term memory. Test-time learned memory. Associative memory. Continual and Online learning.

Recurrence and computation. Recurrent spiking neural networks. Spiking neural networks. Recurrent latent space reasoning. Latent recurrent reasoning. Local attention. Layer normalization in modern recurrent or Post Transformer systems.

Sparsity and interpretability. Sparse positive activations. Monosemantic synapses. Scale-free neural networks. Sparse neural networks.

Architectural directions. Mixture of Experts in the wider Post Transformer landscape, with Jamba as a direct example and BDH as a comparison point. Post Transformer architectures, including BDH, Mamba, xLSTM, RWKV, Jamba, and related systems.

DataForge .pathway x rime

Your Mission

Build one self-contained educational experience for a clearly stated audience. A strong submission lets a learner do all of the following:

- Understand one precise technical claim.
- Manipulate a real concept variable.
- Observe an immediate, meaningful consequence.
- Compare the system output with ground truth, a reference, or a clearly defined expectation.
- Explain the concept back in their own words.
- Understand where the concept appears in BDH or BDH CQ, or how it differs from the corresponding mechanism in BDH.
- Recognize at least one limitation, failure case, or common misconception.

Do not try to explain an entire field. A focused artifact that creates one reliable moment of understanding beats a dashboard with many views and no central lesson.

The one-sentence claim

Before writing code, write one falsifiable sentence that your explainer will teach.

Examples:

- A fixed-shape recurrent state can process a sequence of unbounded duration without allocating a new memory slot for every token, but it can still forget through interference.
- A model can perform repeated latent computation without producing a verbal chain of thought.

The interaction must let the learner reproduce, test, or challenge the sentence. If the artifact cannot in principle reveal that the sentence is wrong, the claim is probably too vague.

The BDH module

Somewhere in the journey, connect your concept to BDH or BDH CQ. Don't tack it on at the end. Weave it into the flow, with its own real learning objective, and ground it in something concrete: an equation, a diagram, a live experiment, or a clearly labeled precomputed result.

Say plainly which system you mean, and why the concept shows up there. Point to what's actually changing, whether that's activity, recurrent state, memory, or trained parameters etc. Work from primary sources, such as [the Dragon Hatchling paper](#) or the [BDH CQ technical report](#), rather than secondhand summaries.

Design standards for the explainer

For inspiration, see [Transformer Explainer](#), [TensorFlow Playground](#), and [Neuronpedia](#), real substrates a learner can manipulate directly.

- **One claim.** Every chart, control, animation, and paragraph should serve the central claim. Cut anything that doesn't.
- **Substrate.** The concept should behave inside the artifact before the learner even acts. A real tiny model, a replay of a real run, or a live toy system all count. A scripted animation doesn't.
- **Visible state.** Shrink the model, sequence, or dataset until the learner can see the variables that matter.
- **Truth beside estimate.** Put the expected answer next to the model output. The gap is often the lesson.
- **Catchy.** Skip the blank canvas and the Run button. Open with a preset already running.
- **Fast feedback.** Controls should respond in under a second. Precompute expensive results and ship them as data.
- **Few controls.** Every control should map to one real variable. Cut decorative sliders.
- **Guide, then sandbox.** Walk the learner through the insight first. Open it up after.
- **No hidden limits.** Caps and approximations are fine. State them.

AI assistance and technical ownership

AI-assisted coding, writing, design, and research are allowed, but teams must understand and defend every component. All AI-generated, reused, or forked work must be disclosed in the README. Forks should add a meaningful contribution. The same applies to the blog, be ready to explain every claim, sentence, and citation.

How judging works

Total score: 100 points, across six criteria.

Technical correctness and depth: 25 points. Judges assess whether the concept, equations, code, measurements, visual mappings, and limitations are accurate. Incorrect claims receive a major penalty.

Technical ownership and live defense: 15 points. Judges assess whether the team understands every major component, can trace the system, can predict the result of changes, and can distinguish real behavior from precomputation or animation.

Learning effectiveness: 15 points. Judges assess the one-sentence claim, audience definition, prerequisites, learning objectives, guided narrative, sixty-second test, and whether a learner can explain the concept after using the artifact.

Interactive substrate and honesty: 15 points. Judges assess whether the concept truly behaves in the artifact, whether controls map to concept variables, whether the important state is visible, whether truth is shown beside estimate, and whether feedback is fast.

BDH or BDH CQ integration and evidence discipline: 10 points. Judges assess whether the BDH module is substantive, technically correct, connected to the selected concept, properly sourced, and explicit about evidence level and limitations.

Craft, robustness, accessibility, and provenance: 10 points. Judges assess visual clarity, writing, mobile usability, loading behavior, link stability, source quality, reproducibility, credits, and licenses.

Blog post: 10 points. Judges assess accuracy, clarity, and whether the post teaches one thing to one reader. Length, polish, and enthusiasm earn nothing.

What to submit

Submit one package containing an explorable learning artifact and its supporting materials.

The artifact itself can be an interactive visual essay, a public web demo, a runnable notebook with interactive controls, a small educational code library, or a hybrid of these. A static article, poster, slide deck, or recorded video by itself is not enough. At least one element must let the learner alter an input, parameter, state, example, or assumption and observe the result. Any animation must be connected to real computation or clearly labeled as a teaching simplification.

The package must include:

- A public artifact URL that opens without sign-in.
- A public source code repository.
- The blog as a pdf file (Talked about below)
- A complete README.
- Clear setup instructions for any notebook or local component.
- At least three recent primary papers from 2022 to 2026 that use, extend, test, or rely on the selected concept, with citations beside technical claims.
- A source and license record for code, data, weights, graphics, fonts, and reused components.
- An AI assistance, code, data, asset, and license disclosure.

The README must explain the claim, the intended learner and prerequisites, the learning objectives, the architecture of the artifact, the role of every major component, which parts are live, precomputed, synthetic, or animated, how to reproduce the results, and credits and licenses.

Accuracy

Do not overclaim or misrepresent any research. Verify claims against primary sources rather than memory or secondhand summaries, and disclose whether a claim is formal or publicly demonstrated.

DataForge .pathway x rime

The blog post

Every team writes one short blog post for a stranger, not a judge. It's part of the submission package, required to be complete. **List of topics are on the last page**, choose one per team, distinct from the main track's approved concepts.

The top 10 blogs from each topic will receive **10 extra points!**

Write:

- Six to eight hundred words, built around one falsifiable idea.
- Written for a reader who knows some ML but not this topic.
- At least two primary papers from 2022 to 2026, cited beside the claims they support.
- At least one limitation, failure case, or open question named.
- No AI slop: no padded sentences that say nothing, no confident claims without a source, no papers that don't exist, no repeating the same few adjectives. Be able to defend every sentence if asked.
- Mention of BDH and related research is mandatory.

What makes it strong:

- A map of how results connect, not just a list of them.
- The mechanism explained in one sentence: why it happens, not just that it happens.
- Specific numbers cited next to the claims they support.
- What didn't work, shown as openly as what did.
- Your own judgment marked clearly and kept separate from the evidence.

Bring in BDH only where it genuinely bears on the topic, at the same accuracy standard as the rest of the track. A forced mention costs marks.

Weak, strong, and exceptional submissions

Weak: generic overviews, paper-summarizing chatbots, static decks, animations passed off as real computation, unchanged forks, a bolted-on BDH mention, unsourced claims, or code the team can't explain.

Strong: one clear claim, a real substrate, honest labeling, a BDH module that teaches, and a disclosed limitation.

Exceptional: a reusable substrate, a claim a learner can reproduce in under a minute, or a resource people keep using after the hackathon.

Topics to choose from (Blog):

Attention and architecture

1. Deriving Attention from Local Interaction Rules
2. Associative Memory in Fast-Weight Architectures
3. Linear Attention Variants and Their Equivalences to Recurrent State Updates
4. Architectural Divergence After the Transformer
5. Fixed-Size Recurrent State vs. Key-Value Caching

Reasoning without tokens

6. Token-Level vs. Continuous-State Intermediate Computation
7. Separability of Linguistic and Non-Linguistic Reasoning
8. Test-Time Compute Allocation Without Autoregressive Decoding
9. Observability Constraints in Latent Reasoning Systems
10. In-Context Learning as Recurrent State Modification

Evaluation and benchmarks

11. Cost–Accuracy Pareto Analysis on Public Reasoning Benchmarks
12. Inference Cost Accounting
13. Strict Task Accuracy vs. Test-Pair Accuracy
14. Parameter-Normalized and Cost-Normalized Benchmark Reporting
15. Skill-Acquisition Efficiency Metrics in the ARC Framework

Compositionality and failure

16. Operator Composition Failures Under Preserved Atomic Performance
17. Object–Property Binding: Diagnostic Task Design
18. Controlled Intervention Ladders for Capability Boundary Localization
19. Demonstration Coverage as a Predictor of Extrapolation Success

Brains, graphs, and memory

20. Sparsity, Locality, and Persistent State
21. Degree Distribution and Modularity in Trained Neural Connectivity Graphs
22. Architecture-Intrinsic Monosemanticity vs. Post-Hoc SAE Decomposition
23. Retrieval Augmentation and Parametric Memory
24. State Persistence Requirements for Long-Horizon Agent Execution